

Contents

Python Cheat Sheet	2
1. Datentypen, Typisierung & Built-ins	2
Wichtige Eigenschaften	2
Wichtige Built-ins & Methoden	2
String-Methoden (Erzeugen immer einen neuen String!)	2
Listen-Methoden (Modifizieren die Liste meist in-place!)	2
Set-Operationen (Mengenlehre)	2
Typisierungskonzepte	2
2. Ablaufsteuerung & Algorithmen	3
KLAUSURFALLE: Mutable Default-Parameter	3
Rekursion	3
Slicing, Comprehensions & Match-Case	3
3. Funktionen, Exceptions & Dateihandling	3
Funktionale Werkzeuge & Generatoren	3
Exceptions (try-except-else-finally)	3
Kontextmanager & CSV-Handling	3
4. Objektorientierte Programmierung (OOP)	4
Klassen, Sichtbarkeiten & UML	4
5. Testing & Binärdaten	4
Embedded C Cheat Sheet	5
1. Build-Prozess & C-Toolchain	5
2. Speicher, Variablen & Pointer	5
Speicherbereiche (Memory Layout)	5
Pointer & Call-by-Reference	5
Zuweisungen bei Pointern (Klausurfalle!)	5
3. Structs, Alignment & Padding	5
Alignment (Speicherausrichtung)	5
KLAUSURFALLE: Structs mit memcmp() vergleichen	6
4. Hardwarenahe Programmierung (Sehr wichtig!)	6
Direkter Register-Zugriff	6
Bitmanipulation (Bitmasking)	6
5. Nebenläufigkeit, Synchronisation & Interrupts	6
Semaphoren & Mutexe	6
Interrupt Service Routinen (ISR)	6

Python Cheat Sheet

Prüfungsrelevante Syntax & Konzepte

1. Datentypen, Typisierung & Built-ins

■ Wichtige Eigenschaften

- **Mutable (Veränderbar):** Objekt kann in-place modifiziert werden (gleiche Speicheradresse).
- **Iterable:** Kann in einer Schleife durchlaufen werden (`__iter__`).

Typ	Mutable	Iterable	Beispiele / Notes
int / float / bool	Nein	Nein	42, 3.14, True
NoneType	Nein	Nein	Repräsentiert Leere / Nullwert
list	Ja	Ja	[1, "a", [2]]
tuple	Nein	Ja	(1, 2) (kann mutable Elemente enthalten!)
str	Nein	Ja	"Text"
range	Nein	Ja	range(start, stop, step) (stop ist exklusiv)
dict	Ja	Ja (Keys)	{"id": 1} -> Keys MÜSSEN immutable sein!
set	Ja	Ja	{1, 2} -> Elemente MÜSSEN immutable sein!

■ Wichtige Built-ins & Methoden

- `sum(iter, start=0)`: Summiert alle numerischen Elemente.
- `enumerate(iter)`: Liefert (Index, Wert).
- `zip(*iters)`: Aggregiert parallel zu Tupeln. Bricht ab, wenn kürzestes Iterable zu Ende.
- `sorted(iter, reverse=True)`: Gibt **neue** sortierte Liste zurück (`list.sort()` modifiziert in-place).
- **F-Strings (Float-Formatierung)**: `f"Wert: {pi:.2f}"` -> 3.14 (Rundet auf 2 Nachkommastellen).

■ String-Methoden (Erzeugen immer einen neuen String!)

- `.strip()`: Entfernt Whitespaces am Anfang und Ende.
- `.split(sep)`: Zerlegt den String am Trennzeichen `sep` in eine Liste.
- `sep.join(iter)`: Verbindet eine Liste von Strings mit dem Trennzeichen `sep`.
- `.replace(old, new)`: Ersetzt Vorkommen von `old` durch `new`.
- `.lower()` / `.upper()`: Transformiert in Klein- bzw. Großbuchstaben.
- `.startswith(prefix)` / `.endswith(suffix)`: Liefert True oder False.

■ Listen-Methoden (Modifizieren die Liste meist in-place!)

- `.append(x)`: Hängt das Element `x` hinten an die Liste an.
- `.extend(iter)`: Hängt alle Elemente des Iterables `iter` an die Liste an.
- `.insert(index, x)`: Fügt `x` an der Position `index` ein.
- `.pop(index=-1)`: Entfernt das Element am `index` (Standard: letztes) und gibt es zurück.
- `.remove(x)`: Entfernt das **erste** Vorkommen von `x` (wirft `ValueError`, wenn nicht existent).
- `.clear()`: Entfernt alle Elemente aus der Liste.
- `.sort(key=None, reverse=False)`: Sortiert die Original-Liste (Gegensatz zu `sorted()`).

■ Set-Operationen (Mengenlehre)

Sets filtern Duplikate automatisch heraus. Logische Verknüpfungen (Bsp: `a = {1, 2}`, `b = {2, 3}`):

- **Vereinigung (Union)**: `a | b` -> {1, 2, 3} (Elemente aus `a` oder `b`)
- **Schnittmenge (Intersection)**: `a & b` -> {2} (Elemente, die in `a` UND `b` sind)
- **Differenz (Difference)**: `a - b` -> {1} (Elemente in `a`, die NICHT in `b` sind)
- **Sym. Differenz (XOR)**: `a ^ b` -> {1, 3} (Elemente, die in genau einem der Sets sind)

■ Typisierungskonzepte

- **Duck Typing**: Typ egal, Hauptsache Objekt hat benötigte Methode ("quakt wie eine Ente").

- **EAFP vs. LBYL**: EAFP (einfach ausführen, Fehler mit try/except fangen) ist Python-Standard. LBYL (vorher mit if prüfen) begünstigt Race Conditions.

2. Ablaufsteuerung & Algorithmen

■ KLAUSURFALLE: Mutable Default-Parameter

Default-Argumente werden nur **einmalig** bei Funktionsdefinition ausgewertet. Niemals mutbare Typen (wie `[]`) als Default setzen!

```
# FALSCH: Teilt die Liste über alle Aufrufe hinweg!
def append_to(element, target=[]):
    target.append(element)
```

```
# RICHTIG:
def append_to_clean(element, target=None):
    if target is None: target = []
    target.append(element)
```

■ Rekursion

Zwei zwingende Komponenten für jede rekursive Funktion:

1. **Basisfall**: Abbruchbedingung (z.B. `if n <= 1: return 1`), verhindert Endlosschleife.
2. **Rekursionsschritt**: Selbstaufruf mit modifiziertem, verkleinertem Problem.

■ Slicing, Comprehensions & Match-Case

- **Slicing**: `sequence[start:stop:step]` (stop ist exklusiv!). Invertieren: `liste[::-1]`
- **List Comprehension**: `[expr for item in iter if cond]`
- **Match-Case (ab 3.10)**:

```
match kommando:
    case "start" | "run": return "Startet" # OR-Verknüpfung
    case ["move", direction]: return direction # Sequenz-Entpacken
    case _: return "Default / Wildcard"
```

3. Funktionen, Exceptions & Dateihandling

■ Funktionale Werkzeuge & Generatoren

- **Generatoren**: Nutzen `yield` statt `return`. Erlauben träge Auswertung (**Lazy Evaluation**). Berechnen Werte erst bei Bedarf (spart RAM).
- **Lambda**: Anonyme Einzeiler: `lambda x, y: x + y`
- `map(func, iter)`: Wendet Funktion auf alle Elemente an.
- `filter(cond, iter)`: Behält nur Elemente, bei denen die Bedingung `True` liefert.
- `reduce(func, iter)`: Reduziert Iterable schrittweise auf Endwert (braucht `functools`).
- `*args` / `**kwargs`: Nimmt variable Argumente als **Tupel** (`args`) bzw. **Dict** (`kwargs`).

■ Exceptions (try-except-else-finally)

```
try:
    wert = int("abc") # Wirft ValueError
except ValueError as e:
    print("Falscher Typ!")
except Exception as e:
    print("Fängt alle restlichen Fehler")
else:
    print("Läuft NUR, wenn KEIN Fehler auftrat")
finally:
    print("Läuft IMMER (gut für Aufräumarbeiten/close)")
```

■ Kontextmanager & CSV-Handling

```
# CSV Schreiben (ohne csv-Modul)
daten = [{"ID", "Name"}, [1, "Alice"], [2, "Bob"]]

with open("daten.csv", "w", encoding="utf-8") as f:
```

```

for zeile in daten:
    # Alle Elemente in Strings umwandeln und mit Semikolon verbinden
    zeile_string = ";".join(str(element) for element in zeile)
    f.write(zeile_string + "\n")

# CSV Lesen (ohne csv-Modul)
with open("daten.csv", "r", encoding="utf-8") as f:
    for zeile in f:
        # Zeilenumbruch (.strip) entfernen und am Semikolon trennen
        werte = zeile.strip().split(";")
        print(werte)

```

4. Objektorientierte Programmierung (OOP)

■ Klassen, Sichtbarkeiten & UML

- **Public (name):** Überall les/schreibbar (UML: +)
- **Protected (_name):** Nur Konvention, nicht strikt (UML: #)
- **Private (__name):** Aktiviert **Name Mangling** -> wird intern zu `_Klasse__name`. Direkter Zugriff von außen wirft `AttributeError`! (UML: -)
- **UML-Beziehungen:** Leere, geschlossene Pfeilspitze = **Vererbung**. Einfache Linie = **Assoziation**.

```

from abc import ABC, abstractmethod

class Konto(ABC): # Abstrakte Basisklasse
    bank_name = "Sparkasse" # Klassenattribut (statisch, geteilt von allen)

    def __init__(self, inhaber):
        self.inhaber = inhaber # Public (+)
        self.__saldo = 0 # Private (-)

    @property
    def saldo(self): # GETTER (Aufruf ohne Klammern: k.saldo)
        return self.__saldo

    @saldo.setter
    def saldo(self, wert): # SETTER (Zuweisung: k.saldo = 100)
        if wert < 0: raise ValueError()
        self.__saldo = wert

    @abstractmethod
    def sprich(self): pass # MUSS in Subklasse überschrieben werden

    @classmethod
    def von_string(cls, string): # Factory (erhält Klasse als 'cls')
        return cls(string)

    @staticmethod
    def info(): # Braucht weder self noch cls
        return "Bank-Info"

```

5. Testing & Binärdaten

- **TDD Zyklus:** Red (Test schlägt fehl) -> Green (Test klappt) -> Refactor (Code aufräumen).
- **Pytest Error Check:** with `pytest.raises(TypeError):`
- **Binärdateien:** Modus `rb` (Lesen) oder `wb` (Schreiben).
 - bytes: **Immutable** (`b"Hello"`)
 - bytearray: **Mutable** (`bytearray(b"Hello")`)

Embedded C Cheat Sheet

Prüfungsrelevante Syntax, Speicher & Hardware

1. Build-Prozess & C-Toolchain

Werkzeug	Eingabeformat	Ausgabeformat
C-Präprozessor	C-Code (.c, .h)	Erweiterter C-Code (Makros aufgelöst)
C-Compiler	Erweiterter C-Code	Assemblercode (.s oder .asm)
Assembler	Assemblercode	Maschinencode / Object-Code (.o)
Linker	Maschinencode (.o + Libs)	Ausführbare Datei (.elf, .bin, .exe)

2. Speicher, Variablen & Pointer

■ Speicherbereiche (Memory Layout)

- **Stack:** Für lokale Variablen, Funktionsaufrufe, Parameter. Wird automatisch verwaltet.
- **Heap:** Für dynamischen Speicher (via `malloc()`, `calloc()`). Muss manuell mit `free()` freigegeben werden (sonst: **Memory Leak**).

■ Pointer & Call-by-Reference

Pointer speichern Speicheradressen. Wichtig für die Rückgabe mehrerer Werte aus einer Funktion (Call-by-Reference).

- `&variable`: "Adresse-von" Operator (gibt den Pointer zurück).
- `*pointer`: Dereferenzierungs-Operator (greift auf den Wert an der Adresse zu).

// Funktion mit Call-by-Reference (Ergebnis über Pointer zurückgeben)

```
int div_int(int a, int b, int *res) {
    if (b == 0) return 1; // Fehlercode
    *res = a / b;          // Wert in Speicher schreiben
    return 0;             // SUCCESS
}
```

// Aufruf in main()

```
int result;
if (div_int(10, 2, &result) == 0) {
    // result ist jetzt 5
}
```

■ Zuweisungen bei Pointern (Klausurfalle!)

```
struct data *old = malloc(sizeof(struct data));
struct data *new = malloc(sizeof(struct data));
```

`old = new;` // FALSCH! Pointer wird umgebogen. Speicher von 'old' ist verloren (Leak), 'new' wird später doppelt befreit!

`*old = *new;` // RICHTIG! Kopiert den tatsächlichen INHALT der Struct von 'new' nach 'old'.

3. Structs, Alignment & Padding

■ Alignment (Speicherausrichtung)

- **Definition:** Variablen werden an Adressen abgelegt, die ein Vielfaches ihrer Größe sind (32-Bit/4-Byte-Variablen an Adressen wie `0x...00`, `0x...04`, `0x...08`).
- **Padding:** Der Compiler fügt unsichtbare "Füllbytes" in Structs ein, um das Alignment der nachfolgenden Variablen sicherzustellen.

```
typedef struct {
    uint8_t temp; // 1 Byte
    // --- 3 Bytes Padding ---
    uint32_t press; // 4 Bytes (Muss auf 4-Byte-Grenze liegen!)
```

```
uint8_t humid; // 1 Byte
// --- 3 Bytes Padding am Ende (für Array-Alignment) ---
} sensor_data_t;
// sizeof(sensor_data_t) = 12 Bytes! (Nicht 6)
```

■ KLAUSURFALLE: Structs mit memcmp() vergleichen

memcmp(a, b, sizeof(struct)) vergleicht den Speicher Byte für Byte. Da die Padding-Bytes uninitialisiert sind und zufälligen "Müll" enthalten können, schlägt memcmp oft fehl, selbst wenn die eigentlichen Daten (temp, press, humid) identisch sind!

Lösung: Immer Element für Element manuell mit if (a->temp == b->temp) vergleichen.

4. Hardwarenahe Programmierung (Sehr wichtig!)

■ Direkter Register-Zugriff

Um Hardware-Register zu beschreiben, muss eine feste Hex-Adresse in einen Pointer gecastet und dereferenziert werden. **Zwingend nötig:** Das Schlüsselwort volatile. Es verbietet dem Compiler, Lese-/Schreibzugriffe wegzuroptimieren.

```
// Schreibe den 16-Bit-Wert 0xFFFF an die Adresse 0x00F3B404
*((volatile uint16_t *)0x00F3B404) = 0xFFFF;
```

```
// Lese einen 32-Bit-Wert von einer Adresse
uint32_t val = *((volatile uint32_t *)0x40001000);
```

■ Bitmanipulation (Bitmasking)

Standard-Aufgabe in Embedded C: Einzelne Bits in einem Register verändern, **ohne** die anderen anzufassen. Sei N die Bit-Position (0 = LSB).

- **Bit Setzen (1):** REG |= (1 << N);
- **Bit Löschen (0):** REG &= ~(1 << N);
- **Bit Umschalten (Toggle):** REG ^= (1 << N);
- **Bit Prüfen:** if (REG & (1 << N)) { ... }

5. Nebenläufigkeit, Synchronisation & Interrupts

■ Semaphore & Mutexe

Werden genutzt, um Ressourcen (z.B. Variablen, Hardware) vor gleichzeitigem Zugriff mehrerer Tasks/Threads zu schützen.

- **Mutex (Mutual Exclusion):** Erfordert 1 binäre Semaphore. (Sperrt den kritischen Bereich).
- **Producer-Consumer-Modell:** Erfordert mindestens 2 (meist 3) Semaphore:
 1. Zählsemaphore für freie Plätze (Free Space).
 2. Zählsemaphore für verfügbare Elemente (Items).
 3. Mutex (Schutz des Puffers/Arrays).

■ Interrupt Service Routinen (ISR)

- Eine ISR unterbricht das Hauptprogramm bei Hardware-Ereignissen (z.B. Timer, Button).
- **Wichtige Regeln für ISRs:**
 1. So kurz und schnell wie möglich ausführen.
 2. Keine blockierenden Aufrufe (kein printf, malloc, delay).
 3. Globale Variablen, die in der ISR geändert und in der main() gelesen werden, **MÜSSEN** als volatile deklariert sein!